



Assignment 2

Red Black Tree

Aim: To design & implement C++ program for a Red Black Tree (a self balancing BST). The program handles node insertion while maintaining Red Black Tree properties through rotations and recoloring.

Algorithm:

1] Insertion (root, key)

- Insert key in root according to BST with Red colour.
- if $\text{currnode} = \text{root}$ colour = Black
- while ($\text{curr} \rightarrow \text{parent} \neq \text{Null}$ & colour of parent = Red)
 - * find parent and grand parent
 - * case 1: (uncle Red) $\{ \text{parent, uncle} = \text{Black}$
 $\text{grandparent} = \text{Red}$
 $\text{curr} = \text{grandparent}$ (recheck)
 - * case 2: (uncle Black)
 - perform rotations relative to parent
LR or RL if node forms Zig Zag
 - do Single Rotation (left, right) on grandparent
 - swap colours of original parent & grandparent
- * root = black

2] Preorder Traversal (curr)

- if $\text{curr} = \text{null}$ return
- print $\text{curr} \rightarrow \text{val}$ and $\text{curr} \rightarrow \text{colour}$
- preorder ($\text{curr} \rightarrow \text{left}$) } if exists
- preorder ($\text{curr} \rightarrow \text{right}$) }



COEP TECHNOLOGICAL UNIVERSITY, PUNE

Wellesley Road, Shivajinagar, Pune-411 005

Conclusion:

Successfully implemented node insertion in Red Black tree with holding its properties.

properties:

Red Black tree is similar to BST w.r.t inserting values in tree.

Condition:

- ① Root is always black.
- ② No adjacent nodes should be both red.
- ③ any path starting from node till same level will have same number of Black Nodes.
- ④ All leaf nodes are pointing to NULL will colour Black.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 typedef enum { RED, BLACK } Color;
5
6 typedef struct Node {
7     int val;
8     Color color;
9     struct Node* left;
10    struct Node* right;
11    struct Node* parent;
12 } Node;
13
14 typedef struct RBTree {
15     Node* root;
16     Node* NIL; // Sentinel leaf node
17 } RBTree;
18
19 // --- Node & Tree Utilities ---
20
21 Node* create_node(RBTree* tree, int val) {
22     Node* new_node = (Node*)malloc(sizeof(Node));
23     if (!new_node) return NULL;
24
25     new_node->val = val;
26     new_node->color = RED;
27     new_node->left = tree->NIL;
28     new_node->right = tree->NIL;
29     new_node->parent = tree->NIL;
30     return new_node;
31 }
32
33 RBTree* create_tree() {
34     RBTree* tree = (RBTree*)malloc(sizeof(RBTree));
35     tree->NIL = (Node*)malloc(sizeof(Node));
36     tree->NIL->color = BLACK;
37     tree->NIL->val = 0;
38     tree->NIL->left = NULL;
39     tree->NIL->right = NULL;
40     tree->NIL->parent = NULL;
41     tree->root = tree->NIL;
42     return tree;
43 }
44
45 void free_tree_helper(RBTree* tree, Node* node) {
46     if (node == tree->NIL) return;
47     free_tree_helper(tree, node->left);
48     free_tree_helper(tree, node->right);
49     free(node);
50 }
51
52 void free_tree(RBTree* tree) {
53     free_tree_helper(tree, tree->root);
54     free(tree->NIL);

```

```

50 }
51
52 void free_tree(RBTree* tree) {
53     free_tree_helper(tree, tree->root);
54     free(tree->NIL);
55     free(tree);
56 }
57
58 // --- Rotations ---
59
60 void left_rotate(RBTree* tree, Node* x) {
61     Node* y = x->right;
62     x->right = y->left;
63
64     if (y->left != tree->NIL)
65         y->left->parent = x;
66
67     y->parent = x->parent;
68
69     if (x->parent == tree->NIL)
70         tree->root = y;
71     else if (x == x->parent->left)
72         x->parent->left = y;
73     else
74         x->parent->right = y;
75
76     y->left = x;
77     x->parent = y;
78 }
79
80 void right_rotate(RBTree* tree, Node* y) {
81     Node* x = y->left;
82     y->left = x->right;
83
84     if (x->right != tree->NIL)
85         x->right->parent = y;
86
87     x->parent = y->parent;
88
89     if (y->parent == tree->NIL)
90         tree->root = x;
91     else if (y == y->parent->right)
92         y->parent->right = x;
93     else
94         y->parent->left = x;
95
96     x->right = y;
97     y->parent = x;
98 }
99
100 // --- Insertion & Fixing Properties ---
101
102 void insert_fixup(RBTree* tree, Node* z) {
103     while (z->parent->color == RED) {

```

```

102 void insert_fixup(RBTree* tree, Node* z) {
103     while (z->parent->color == RED) {
104         if (z->parent == z->parent->parent->left) {
105             Node* y = z->parent->parent->right; // Uncle
106
107             if (y->color == RED) { // Case 1: Uncle is Red
108                 z->parent->color = BLACK;
109                 y->color = BLACK;
110                 z->parent->parent->color = RED;
111                 z = z->parent->parent;
112             } else {
113                 if (z == z->parent->right) { // Case 2: Triangle -> Line
114                     z = z->parent;
115                     left_rotate(tree, z);
116                 }
117                 // Case 3: Line configuration
118                 z->parent->color = BLACK;
119                 z->parent->parent->color = RED;
120                 right_rotate(tree, z->parent->parent);
121             }
122         } else { // Symmetric cases for right parent
123             Node* y = z->parent->parent->left; // Uncle
124
125             if (y->color == RED) { // Case 1
126                 z->parent->color = BLACK;
127                 y->color = BLACK;
128                 z->parent->parent->color = RED;
129                 z = z->parent->parent;
130             } else {
131                 if (z == z->parent->left) { // Case 2
132                     z = z->parent;
133                     right_rotate(tree, z);
134                 }
135                 // Case 3
136                 z->parent->color = BLACK;
137                 z->parent->parent->color = RED;
138                 left_rotate(tree, z->parent->parent);
139             }
140         }
141     }
142     tree->root->color = BLACK;
143 }
144
145 void insert(RBTree* tree, int data) {
146     Node* z = create_node(tree, data);
147     Node* y = tree->NIL;
148     Node* x = tree->root;
149
150     while (x != tree->NIL) {
151         y = x;
152         if (z->val < x->val)
153             x = x->left;
154         else if (z->val > x->val)
155             x = x->right;

```

```

152     if (z->val < x->val)
153         x = x->left;
154     else if (z->val > x->val)
155         x = x->right;
156     else {
157         // Duplicate value found: discard
158         free(z);
159         return;
160     }
161 }
162
163 z->parent = y;
164 if (y == tree->NIL)
165     tree->root = z;
166 else if (z->val < y->val)
167     y->left = z;
168 else
169     y->right = z;
170
171 insert_fixup(tree, z);
172 }
173
174 // --- Traversals ---
175
176 void preorder_traversal(RBTree* tree, Node* root) {
177     if (root == tree->NIL)
178         return;
179
180     printf("%d(%s) ", root->val, root->color == RED ? "RED" : "BLACK");
181     preorder_traversal(tree, root->left);
182     preorder_traversal(tree, root->right);
183 }
184
185 // --- Main Executable ---
186
187 int main() {
188     RBTree* tree = create_tree();
189
190     // Inserting integers into Red-Black Tree
191     insert(tree, 10);
192     insert(tree, 20);
193     insert(tree, 30);
194     insert(tree, 15);
195     insert(tree, 25);
196     insert(tree, 5);
197     insert(tree, 1);
198
199     printf("Preorder Traversal (Node Value and Color):\n");
200     preorder_traversal(tree, tree->root);
201     printf("\n");
202
203     free_tree(tree);
204     return 0;
205 }

```